C_Programming_Assignment_4

Name : Baranitha A 1965260049

Task 1: Data Design Review

Q1.1 – Diagnose the Design

Identify at least five design deficiencies and explain what could go wrong in the health centre.

1. Parallel Arrays: Patient details are stored in separate arrays. This may cause data mismatching when sorting or updating patient records.
2. Phone Number Storage: char phone[100][10] cannot store a 10-digit phone number along with the null character. This may cause data truncation.
3. Blood Pressure Storage: Blood pressure is stored in a single float variable. It should be divided into systolic and diastolic values for accurate recording.
4. Date Stored as Text: The visit date is stored as an 8-character string. This makes date validation and comparison difficult.
5. No Limit Check: There is no check to ensure that the patient count stays below 100. This may cause memory errors.
6. Global Variables: Global arrays can be accessed and modified by different functions, increasing the risk of accidental changes.
7. Difficult Sorting: Sorting patients by age is difficult because all related arrays must be rearranged together.

Q1.2 – Repair the Design

Redesign the data using structures, including nested structures for date and blood pressure.

Program:

```
#include <stdio.h>

#include <string.h>

struct date {

   int day, month, year;

};

struct bp {

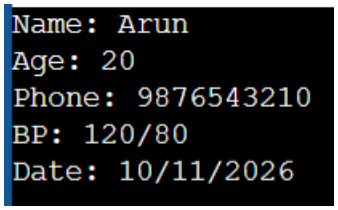
   int systolic, diastolic;

};

struct patient {
```

```
    char name[20];
    int age;
    char phone[11];
    struct bp pressure;
    struct date visit;
};
void addPatient(struct patient *p, char n[], int a,
                char ph[], int s, int d, int day, int m, int y) {
    strcpy(p->name, n);
    p->age = a;
    strcpy(p->phone, ph);
    p->pressure.systolic = s;
    p->pressure.diastolic = d;
    p->visit.day = day;
    p->visit.month = m;
    p->visit.year = y;
}
int main() {
    struct patient p;
    addPatient(&p, "Arun", 20, "9876543210",
               120, 80, 10, 11, 2026);
    printf("Name: %s\n", p.name);
    printf("Age: %d\n", p.age);
    printf("Phone: %s\n", p.phone);
    printf("BP: %d/%d\n", p.pressure.systolic,
           p.pressure.diastolic);
    printf("Date: %02d/%02d/%d\n",
           p.visit.day, p.visit.month, p.visit.year);
    return 0;
}
```

Expected Output:



Q1.3 – Design Defence

The structure-based design stores all patient details together. It makes sorting easier because the entire patient record can be moved at once. Structures can be passed to functions using pointers, reducing unnecessary copying. New fields can also be added easily without modifying multiple arrays.

Alternative Concept and Complexity Analysis

1. When are parallel arrays preferred?

Parallel arrays are preferred when only individual fields need processing or when working with large datasets where accessing one particular field is more important.

2. Sorting comparison:

- Array of structures: Each swap moves the entire patient record.
- Parallel arrays: Each swap requires rearranging every related array.
- Both generally take $O(n^2)$ time using simple sorting.
- Array of structures may use extra padding bytes for memory alignment.
- Parallel arrays may use less padding but require careful synchronization.

Task 2: From Requirement to Nested Structures

Q2.1 – Identify the Entities

The main entities and their fields are:

1. Parcel: Parcel ID, weight, booking date, service type and charge.
2. Sender: Name, city and PIN code.
3. Receiver: Name, city and PIN code.
4. Address: City and PIN code.
5. Date: Day, month and year.

Sender, receiver, address and date structures are nested inside the parcel structure.

Q2.2 – Define the Structures

C Program:

```
#include <stdio.h>
typedef struct {
    int day, month, year;
} Date;
typedef struct {
    char name[30], city[20];
    int pin;
} Person;
typedef struct {
    int id;
    float weight, charge;
    Date booking;
    Person sender, receiver;
    char service[10];
} Parcel;
int main() {
    Parcel list[3];
    return 0;
}
```

Q2.3 – Design the Operations

(a) Sorting parcels by weight:

1. Start from the first parcel.
2. Find the parcel with the smallest weight.
3. Swap it with the current parcel.
4. Repeat until all parcels are sorted.

(b) Listing parcels by receiver city:

1. Read the required city.
2. Check the receiver city of every parcel.

3. If it matches, display the parcel details.
4. Repeat until all parcels are checked.

Q2.4 – Access Paths

(i) Using array and dot operator:

list[2].to.pincode

(ii) Using structure pointer and arrow operator:

p->to.pincode

Here, p points to the third parcel.

Complete the Code – TODO 1 and TODO 2

Source Code:

```
#include <stdio.h>
struct date {
    int day, month, year;
};
struct address {
    char city[20];
    int pincode;
};
struct parcel {
    int id;
    float weight;
    struct address to;
    struct date booked;
};
void print_parcel(const struct parcel *p) {
    printf("%d %.1f kg %s-%d %02d/%02d/%d\n",
    p->id, p->weight, p->to.city, p->to.pincode,
    p->booked.day, p->booked.month,
    p->booked.year);
}
```

```
void sort_by_weight(struct parcel p[], int n) {
    int i, j, min;
    struct parcel temp;
    for (i = 0; i < n - 1; i++) {
        min = i;
        for (j = i + 1; j < n; j++) {
            if (p[j].weight < p[min].weight)
                min = j;
        }
        temp = p[i];
        p[i] = p[min];
        p[min] = temp;
    }
}

int main() {
    struct parcel list[3] = {
        {501, 4.5f, {"Madurai", 625001}, {12, 10, 2026}},
        {502, 1.2f, {"Chennai", 600077}, {12, 10, 2026}},
        {503, 2.8f, {"Coimbatore", 641001}, {13, 10, 2026}}
    };

    sort_by_weight(list, 3);

    for (int i = 0; i < 3; i++)
        print_parcel(&list[i]);

    return 0;
}
```

Expected Output:

```
502 1.2 kg Chennai-600077 12/10/2026
503 2.8 kg Coimbatore-641001 13/10/2026
501 4.5 kg Madurai-625001 12/10/2026
```

Alternative Concept and Complexity Analysis

Flat Structure vs Nested Structure:

- Nested structure: Address and date are stored as separate structures inside parcel.
- Flat structure: All address and date fields are directly stored inside parcel.
- Nested structures allow reuse of address and date types.
- Flat structures have simpler access paths.
- Both have similar memory requirements, but actual size depends on padding and alignment.

Number of bytes copied in each swap:

sizeof(struct parcel)

Each swap moves one complete parcel structure.

Task 3: Unions and Memory Layout

Q3.1 – Structure or Union?

Structure Design:

```
struct packet {
    float temperature;
    int humidity;
    char status[8];
};
```

Tagged Union Design:

```
struct reading {
    enum kind type;
    union {
        float celsius;
        int percent;
        char status[8];
```

```
    } value;
};
```

Justification:

A structure stores all three values separately, whereas a union shares the same memory for all members. Since the weather station sends only one type of value at a time, a tagged union saves memory.

Q3.2 – Predict the Size

Design	Expected Size
Structure	16 bytes
Union	8 bytes
Tagged union structure	12 bytes

Explanation:

- The structure stores memory for all members.
- The union uses memory equal to its largest member.
- The tagged union contains a 4-byte type tag and an 8-byte union.
- Padding is added for memory alignment.

Q3.3 – Implement and Test

Source Code:

```
#include <stdio.h>
enum kind { TEMPERATURE, HUMIDITY, STATUS };
struct reading {
    enum kind type;
    union {
        float celsius;
        int percent;
        char status[8];
    } value;
};
void show(const struct reading *r) {
    switch (r->type) {
        case TEMPERATURE:
```

```
            printf("Temp %.1f C\n", r->value.celsius);
            break;
        case HUMIDITY:
            printf("Humidity %d %%\n", r->value.percent);
            break;
        case STATUS:
            printf("Status %s\n", r->value.status);
            break;
    }
}

int main() {
    struct reading log[3] = {
        {TEMPERATURE, {.celsius = 31.5f}},
        {HUMIDITY, {.percent = 68}},
        {STATUS, {.status = "OK"}}
    };
    printf("sizeof union = %zu, sizeof struct reading = %zu\n",
           sizeof log[0].value, sizeof log[0]);
    for (int i = 0; i < 3; i++)
        show(&log[i]);
    return 0;
}
```

Expected Output:

```
sizeof union = 8, sizeof struct reading = 12
Temp 31.5 C
Humidity 68 %
Status OK
```

Q3.4 – Misuse Experiment

Experiment: Store temperature and then read the humidity member.

```
struct reading r;

r.type = TEMPERATURE;

r.value.celsius = 31.5;


printf("%d", r.value.percent);
```

Explanation:

The output may be meaningless because the union shares memory between its members. Reading a member other than the one last stored does not give a valid humidity value.

Why is the Structure 12 Bytes?

The structure contains:

- Enum tag – 4 bytes.
- Union – 8 bytes.
- Total – 12 bytes.

No additional padding is required in this example.

Alternative Concept and Complexity Analysis

1. Separate Arrays:

- Store temperature, humidity and status in separate arrays.
- Access is fast when the packet type is already known.
- Requires separate indexing and management.

2. Dynamic Memory with void Pointer:

- Allocate memory for each packet separately.
- Use a void pointer to refer to the data.
- Requires additional memory allocation and type handling.

Comparison:

- Tagged union uses fixed memory and provides easy access.
- Separate arrays are useful when processing one packet type at a time.
- Dynamic allocation is flexible but requires extra memory and management.
- Tagged union provides efficient memory usage for this application.

Task 4: Structure Forensics

Q4.1 – Locate the Defects

1. Pass by Value: The function receives a copy of the structure. Therefore, changes to the balance do not affect the original account.
2. Incorrect Pointer Access: The statement p.number and p.balance is incorrect because p is a pointer. The arrow operator (->) must be used.

Q4.2 – Construct Evidence

Program:

```
#include <stdio.h>
struct account {
    int number;
    float balance;
};
void addInterest(struct account a, float rate) {
    printf("Function address: %p\n", (void*)&a);
    a.balance += a.balance * rate / 100;
}
int main() {
    struct account list[3] = {
        {101, 5000}, {102, 12000}, {103, 800}
    };
    printf("Main address: %p\n", (void*)&list[0]);
    addInterest(list[0], 4);
    return 0;
}
```

Expected Output:

```
Main address: 0x7ffc60f10550
Function address: 0x7ffc60f10538
```

Q4.3 – Repair the Program

Corrected Source Code:

```
#include <stdio.h>

struct account {
    int number;
    float balance;
};

void addInterest(struct account *a, float rate) {
    a->balance += a->balance * rate / 100;
}

int main() {
    struct account list[3] = {
        {101, 5000}, {102, 12000}, {103, 800}
    };

    struct account *p = list;

    for (int i = 0; i < 3; i++)
        addInterest(&list[i], 4);

    for (int i = 0; i < 3; i++)
        printf("%d %.2f\n", p[i].number, p[i].balance);
    return 0;
}
```

Expected Output:

```
101 5200.00
102 12480.00
103 832.00
```

Q4.4 – Prevent Recurrence

- Pass by Value: Used when the original structure should not be modified.
- Pass by Pointer: Used when the function needs to modify the original structure.
- Pass by Const Pointer: Used to access the structure without modifying it.

Alternative Concept and Complexity Analysis

Instead of passing a pointer, the function can return the modified structure.

```
struct account addInterest(struct account a, float rate) {
   a.balance += a.balance * rate / 100;
   return a;
}
```

Call it using:

```
list[i] = addInterest(list[i], 4);
```

Comparison:

- Pass by value copies the entire structure.
- Pass by pointer passes only the address.
- Pointer method uses less memory and avoids unnecessary copying.
- Both methods take O(1) time for a single account.

Task 5: Storage Classes – The Same Counter Four Ways

Q5.1 – Four Implementations

(a) Automatic Variable

```
int auto_token(void) {
   int token = 1000;
   return ++token;
}
```

(b) Static Variable

```
int next_token(void) {
   static int token = 1000;
   return ++token;
}
```

(c) Global Variable Using extern

File 1 – main.c

```
#include <stdio.h>
int token = 1000;
int next_token(void);

int main() {
   printf("%d %d %d", next_token(),
         next_token(), next_token());
   return 0;
}
```

File 2 – token.c

```
extern int token;

int next_token(void) {
   return ++token;
}
```

(d) Register Variable

```
for (register int i = 1; i <= 3; i++)
   printf("%d\n", i);
```

Outputs:

Method	Output
Automatic	1001 1001 1001
Static	1001 1002 1003
Global	1001 1002 1003
Register	1 2 3

Q5.2 – Change the Requirement

To support separate veg and non-veg counters:

- Automatic: Create two separate local variables.
- Static: Use two static variables.

- Global: Create two global variables.
- Register: Use separate loop variables for each counter.

Static and global variables require separate counters to avoid interference.

Q5.3 – Compare the Storage Classes

Storage class	Scope	Lifetime	Initial value
Automatic	Local	Function execution	Garbage value
Static	Local	Entire program	0
Global	File/program	Entire program	0
Register	Local	Function execution	Garbage value

Why is the address of a register variable not allowed?

A register variable is requested to be stored in a CPU register. Therefore, its memory address cannot be accessed using the address operator (&).

Complete the Code – TODO 1

Source Code:

```
#include <stdio.h>

int next_token(void) {
    static int token = 1000;
    return ++token;
}

int auto_token(void) {
    int token = 1000;
    return ++token;
}

int main() {
    int s1 = next_token(), s2 = next_token(), s3 = next_token();
```

```
    int a1 = auto_token(), a2 = auto_token(), a3 = auto_token();

    printf("static: %d %d %d\n", s1, s2, s3);
    printf("auto : %d %d %d\n", a1, a2, a3);

    return 0;
}
```

Input: No input required.

Expected Output:

static: 1001 1002 1003

auto : 1001 1001 1001

Alternative Concept and Complexity Analysis

If the counter is passed using a pointer:

```
int next_token(int *counter) {
    return ++(*counter);
}
```

Comparison:

- Static and global counters retain values throughout the program.
- Pointer-based counters are stored in the caller.
- Pointer-based counters support multiple independent counters easily.
- All methods take O(1) time.
- Pointer-based counters avoid using global variables.

Task 5: Storage Classes – The Same Counter Four Ways

Q5.1 – Four Implementations

(a) Automatic Variable

```
int auto_token(void) {
    int token = 1000;
    return ++token;
}
```

(b) Static Variable

```
int next_token(void) {
   static int token = 1000;
   return ++token;
}
```

(c) Global Variable Using extern

File 1 – main.c

```
#include <stdio.h>
int token = 1000;
int next_token(void);

int main() {
   printf("%d %d %d", next_token(),
         next_token(), next_token());
   return 0;
}
```

File 2 – token.c

```
extern int token;

int next_token(void) {
   return ++token;
}
```

(d) Register Variable

```
for (register int i = 1; i <= 3; i++)
   printf("%d\n", i);
```

Outputs:

Method	Output
Automatic	1001 1001 1001
Static	1001 1002 1003
Global	1001 1002 1003
Register	1 2 3

Q5.2 – Change the Requirement

To support separate veg and non-veg counters:

- Automatic: Create two separate local variables.
- Static: Use two static variables.
- Global: Create two global variables.
- Register: Use separate loop variables for each counter.

Static and global variables require separate counters to avoid interference.

Q5.3 – Compare the Storage Classes

Storage class	Scope	Lifetime	Initial value
Automatic	Local	Function execution	Garbage value
Static	Local	Entire program	0
Global	File/program	Entire program	0
Register	Local	Function execution	Garbage value

Why is the address of a register variable not allowed?

A register variable is requested to be stored in a CPU register. Therefore, its memory address cannot be accessed using the address operator (&).

Complete the Code – TODO 1

Source Code:

```
#include <stdio.h>

int next_token(void) {
    static int token = 1000;
    return ++token;
```

```
}

int auto_token(void) {
    int token = 1000;
    return ++token;
}

int main() {
    int s1 = next_token(), s2 = next_token(), s3 = next_token();

    int a1 = auto_token(), a2 = auto_token(), a3 = auto_token();

    printf("static: %d %d %d\n", s1, s2, s3);
    printf("auto : %d %d %d\n", a1, a2, a3);

    return 0;
}
```

Expected Output:

```
static: 1001 1002 1003
auto : 1001 1001 1001
```

Alternative Concept and Complexity Analysis

If the counter is passed using a pointer:

```
int next_token(int *counter) {
    return ++(*counter);
}
```

Comparison:

- Static and global counters retain values throughout the program.
- Pointer-based counters are stored in the caller.
- Pointer-based counters support multiple independent counters easily.
- All methods take O(1) time.
- Pointer-based counters avoid using global variables.

Task 6: Design Challenge

Stage 1 – Interpret the Problem

Entities and their data:

1. Packaged Items: Item name, price, stock and expiry date.
2. Cooked Items: Item name, price, stock and preparation time.
3. Combo Meals: Combo name, price and up to three items.
4. Sales: Item name, quantity and total amount.

Requirements:

- Record item details and sales.
- Update stock after every sale.
- Warn about items expiring within two days.
- Display the bill summary sorted by revenue.

Stage 2 – Propose the Data Model

Use the following:

- Nested structures for date and time.
- Array of structures to store items.
- Tagged union to represent packaged, cooked and combo items.
- Structure for recording sales.

Stage 3 – Critical Design Decisions

1. Stock must be checked before processing a sale.
2. Stock should decrease after a successful sale.
3. Expiry dates must be checked using a date structure.
4. Static variables can store values that must be retained between function calls.
5. Global variables should be avoided unless shared access is necessary.

Stage 4 – Design the Structures and Functions

C Program:

```
#include <stdio.h>
struct date {
   int day, month, year;
};
```

```
struct item {
    char name[20];
    float price;
    int stock;
    struct date expiry;
};

struct sale {
    char name[20];
    int quantity;
    float revenue;
};

void recordSale(struct item *p, int qty) {
    if (p->stock >= qty) {
        p->stock -= qty;
        printf("Sale successful\n");
    }
    else {
        printf("Insufficient stock\n");
    }
}

int main() {
    struct item a = {"Juice", 30, 10, {12, 11, 2026}};

    struct sale s;

    printf("Item: %s\n", a.name);
```

```
    printf("Price: %.2f\n", a.price);

    recordSale(&a, 2);

    printf("Remaining stock: %d\n", a.stock);

    return 0;
}
```

Function Prototypes:

```
void recordSale(struct item *p, int qty);
void checkExpiry(struct item p[], int n);
void sortRevenue(struct sale s[], int n);
```

Pseudocode – Recording a Sale:

1. Read item and quantity.
2. Check available stock.
3. If stock is sufficient, calculate the amount.
4. Reduce stock.
5. Display the bill.
6. Otherwise, display insufficient stock.

Pseudocode – Expiry Warning:

1. Read the current date.
2. Check each packaged item's expiry date.
3. Find items expiring within two days.
4. Display a warning.

Stage 5 – Implement and Demonstrate

Expected Output:

```
Item: Juice
Price: 30.00
Sale successful
Remaining stock: 8
```

Test Cases:

Test	Input / Condition	Expected Result
1	Sell 2 items	Sale successful
2	Sell 15 items	Insufficient stock
3	Expiry within 2 days	Warning displayed
4	Different sales	Summary sorted by revenue

Alternative Concept and Complexity Analysis

Array vs Linked List:

- Array stores items in continuous memory.
- Linked list stores items using dynamically allocated nodes.
- Adding an item at the beginning of a linked list takes O(1).
- Removing an item takes O(n) when searching is required.
- Searching takes O(n) in both structures.
- Arrays may require shifting elements when inserting or deleting.
- Linked lists require extra memory for next pointers.

Conclusion:

The canteen inventory system uses structures, nested structures and arrays to manage items, sales and stock efficiently. It also helps identify expiring products and generate revenue summaries.